ΥΠΟΛΟΓΙΣΤΙΚΗ ΝΟΗΜΟΣΥΝΗ – ΣΤΑΤΙΣΤΙΚΗ ΜΑΘΗΣΗ

ΧΡΙΣΤΙΝΑ ΚΟΥΜΠΟΓΙΑΝΝΗ

ΑΜ: 140

ΠΕΡΙΕΧΟΜΕΝΑ

ΤΙΤΛΟΣ ΕΡΓΑΣΙΑΣ 2

ΕΠΕΞΗΓΗΣΗ ΒΑΣΙΚΩΝ ΕΝΝΟΙΩΝ: 2

- SVM 2
- How SVMs work 2
- Features 3
- Τύποι SVMs 3
- Τύποι πυρήνων και τα χαρακτηριστικά τους 3
- Τα πλεονεκτήματα των SVMs 4
- Τα μειονεκτήματα των SVMs 5
- PCA 5
- Kernel trick 5
- CIFAR-10: 5

ΥΛΟΠΟΙΗΣΗ ΚΑΙ ΕΠΕΞΗΓΗΣΗ ΚΩΔΙΚΑ **MNIST** 6

ΥΛΟΠΟΙΗΣΗ ΚΑΙ ΕΠΕΞΗΓΗΣΗ ΚΩΔΙΚΑ **CIFAR-10** 13

ΑΝΑΛΥΣΗ ΚΑΙ ΧΡΗΣΗ ΔΙΑΦΟΡΕΤΙΚΩΝ ΠΥΡΗΝΩΝ ΣΤΟΝ ΚΩΔΙΚΑ 17

- LINEAR KERNEL, DIFFERENT VALUES OF C 17
- POLYNOMYAL KERNEL, DIFFERENT VALUES OF C, DEGREE 21
- SIGMOID KERNEL, DIFFERENT VALUES OF C 26
- ΣΥΓΚΡΙΣΗ ΚΑΙ ΕΥΡΕΣΗ ΤΟΥ ΚΑΛΥΤΕΡΟΥ ΠΥΡΗΝΑ ΓΙΑ ΤΟ **MNIST** 29
- ΣΥΓΚΡΙΣΗ ΚΑΛΥΤΕΡΩΝ ΔΟΚΙΜΑΣΜΕΝΩΝ ΠΥΡΗΝΩΝ ΓΙΑ ΤΟ **CIFAR-10** 30

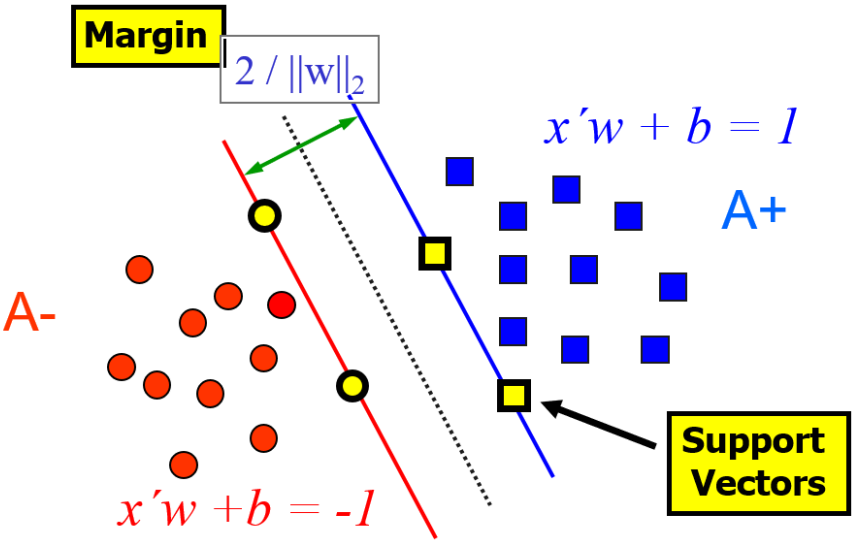
ΤΙΤΛΟΣ ΕΡΓΑΣΙΑΣ:

Να γραφεί πρόγραμμα σε οποιαδήποτε γλώσσα προγραμματισμού το οποίο να υλοποιεί/χρησιμοποιεί ένα Support Vector Machine για πρόβλημα διαχωρισμού κλάσεων.

ΕΠΕΞΗΓΗΣΗ ΒΑΣΙΚΩΝ ΕΝΝΟΙΩΝ:

SVM: Στη μηχανική μάθηση, οι μηχανές διανυσμάτων υποστήριξης (support vector machine - SVM) είναι επιβλεπόμενα μοντέλα **μέγιστου περιθωρίου** με σχετικούς αλγόριθμους εκμάθησης που αναλύουν δεδομένα για ανάλυση ταξινόμησης και παλινδρόμησης. Το SVM σχετίζεται στενά με: Μηχανές πυρήνα (γενίκευση SVM), ταξινομητές μεγάλου περιθωρίου, χώρος Hilbert, διαδικασία Gauss, Boosting. Υπάρχουν συγκεκριμένοι τύποι SVM που χρησιμοποιούνται για συγκεκριμένα προβλήματα μηχανικής μάθησης, όπως η παλινδρόμηση διανύσματος υποστήριξης (SVR) που είναι μια επέκταση της ταξινόμησης διανυσμάτων υποστήριξης (SVC). Είναι μαθηματικές εξισώσεις συντονισμένες για να δώσουν την πιο ακριβή απάντηση όσο το δυνατόν γρηγορότερα.

How SVMs work: Ένας απλός γραμμικός ταξινομητής SVM λειτουργεί κάνοντας μια ευθεία γραμμή μεταξύ δύο κλάσεων. Αυτό σημαίνει ότι όλα τα σημεία δεδομένων στη μία πλευρά της γραμμής θα αντιπροσωπεύουν μια κατηγορία (κλάση) και τα σημεία δεδομένων στην άλλη πλευρά της γραμμής θα τοποθετηθούν σε διαφορετική κατηγορία. Αυτό που κάνει τον **γραμμικό αλγόριθμο SVM** (linear SVM) καλύτερο από ορισμένους από τους άλλους αλγόριθμους, όπως οι k-NN, είναι ότι επιλέγει την καλύτερη γραμμή για την ταξινόμηση των σημείων δεδομένων σας. Επιλέγει τη γραμμή που χωρίζει τα δεδομένα και είναι όσο το δυνατόν πιο μακριά **(max margin)** από τα σημεία δεδομένων διανυσμάτων υποστήριξης. Τα πλησιέστερα σημεία (των διαφορετικών δεδομένων) δίνουν τα διανύσματα υποστήριξης (**support vectors**) που θα χρησιμοποιηθούν για να βρεθεί αυτήν τη γραμμή που θα διαχωρίσει τα δεδομένα σε δύο κλάσεις. Αυτή η γραμμή ονομάζεται όριο απόφασης. Το όριο απόφασης δεν χρειάζεται να είναι μια γραμμή. Αναφέρεται επίσης ως υπερεπίπεδο επειδή μπορεί να βρεθεί το όριο απόφασης με οποιονδήποτε αριθμό χαρακτηριστικών, όχι μόνο με δύο.



Εικόνα 1: support vectors

Features: Στο πλαίσιο των Μηχανών Διανυσμάτων Υποστήριξης (SVM), τα χαρακτηριστικά **(features)** αναφέρονται στις μεμονωμένες μετρήσιμες ιδιότητες ή χαρακτηριστικά των δεδομένων που χρησιμοποιούνται ως είσοδοι για την εκπαίδευση του μοντέλου. Τα χαρακτηριστικά είναι θεμελιώδη στοιχεία του συνόλου δεδομένων και ο ρόλος τους στα SVM είναι κρίσιμος επειδή καθορίζουν τον τρόπο με τον οποίο ο αλγόριθμος βρίσκει το βέλτιστο υπερεπίπεδο για ταξινόμηση ή παλινδρόμηση.

Τύποι SVMs:

- **Simple SVM**: Συνήθως χρησιμοποιείται για προβλήματα γραμμικής παλινδρόμησης και ταξινόμησης.
- **Kernel SVM**: Έχει μεγαλύτερη ευελιξία για μη γραμμικά δεδομένα, επειδή γίνεται να προστεθούν περισσότερα χαρακτηριστικά για να χωρέσουν σε ένα υπερεπίπεδο αντί για έναν δισδιάστατο χώρο (kernel trick).

Τύποι πυρήνων και τα χαρακτηριστικά τους:

- **Γραμμικός** πυρήνας (f(X) = w^T * X + b): Χρησιμοποιεί πρωτότυπα χαρακτηριστικά (καλό για γραμμικά διαχωρίσιμα δεδομένα). Σε αυτήν την εξίσωση, w είναι το διάνυσμα βάρους που χρειάζεται να ελαχιστοποιηθεί, X είναι τα δεδομένα που προσπαθούν να ταξινομηθούν σε κλάσεις και b είναι ο γραμμικός συντελεστής που υπολογίζεται από τα δεδομένα προπόνησης (**train data**). Αυτή η εξίσωση ορίζει το όριο απόφασης που επιστρέφει το SVM. Λειτουργεί πολύ καλά όταν υπάρχουν πολλά χαρακτηριστικά και τα προβλήματα ταξινόμησης π.χ. κειμένου έχουν πολλά χαρακτηριστικά. Οι συναρτήσεις γραμμικού πυρήνα είναι πιο

γρήγορες από τις περισσότερες άλλες και έχουν λιγότερες παραμέτρους για βελτιστοποίηση.

- **Πολυωνυμικός** πυρήνας (f(X1, X2) = (a + X1^T * X2) ^ b): Εισάγει όρους πολυωνυμικής αλληλεπίδρασης μεταξύ χαρακτηριστικών. Δεν είναι τόσο υπολογιστικά αποδοτικός όσο άλλοι πυρήνες και οι προβλέψεις του δεν είναι τόσο ακριβείς. Στην επάνω συνάρτηση, το f(X1, X2) αντιπροσωπεύει το πολυωνυμικό όριο απόφασης που θα διαχωρίσει τα δεδομένα. Τα X1 και X2 αντιπροσωπεύουν τα δεδομένα.
- Πυρήνας **RBF** (Radial Basis Function)
  f(X1, X2) = exp(-gamma * ||X1 - X2||^2): Δημιουργεί χαρακτηριστικά ομοιότητας με βάση την απόσταση από άλλα σημεία του συνόλου δεδομένων. Ένας από τους πιο ισχυρούς και ευρέως χρησιμοποιούμενους πυρήνες σε SVM. Συνήθως η επιλογή για μη γραμμικά δεδομένα. Το gamma ελέγχει πόσο εντοπισμένη (localized) είναι η επιρροή ενός μόνο σημείου στο όριο απόφασης. Το ||X1 - X2|| είναι το βαθμωτό γινόμενο μεταξύ των χαρακτηριστικών.
- **Σιγμοειδής** πυρήνας (f(X, y) = tanh(alpha * X^T * y + C)): Προσαρμόζει τα χαρακτηριστικά με τρόπο παρόμοιο με τα νευρωνικά δίκτυα. Σε αυτή τη συνάρτηση, το άλφα είναι ένα διάνυσμα βάρους και το C είναι μια τιμή μετατόπισης για να ληφθεί υπόψη κάποια εσφαλμένη ταξινόμηση δεδομένων που μπορεί να συμβεί.

Η παράμετρος **gamma** (για RBF και πολυωνυμικούς πυρήνες) ελέγχει πόσο μακριά φτάνει η επιρροή ενός μόνο σημείου δεδομένων. Για χαμηλό gamma, τα σημεία που είναι μακριά το ένα από το άλλο θεωρούνται παρόμοια, με αποτέλεσμα ένα πιο ομαλό όριο απόφασης. Αυτό σημαίνει ότι ακόμη και τα σημεία δεδομένων που απέχουν πολύ μεταξύ τους στο διάστημα θα εξακολουθούν να έχουν κάποια επιρροή το ένα στο άλλο. Ενώ για υψηλό gamma, τα κοντινά σημεία επηρεάζουν το ένα το άλλο, οδηγώντας σε πιο περίπλοκα όρια αποφάσεων. Αυτό σημαίνει ότι ένα σημείο εκπαίδευσης (training point) επηρεάζει μόνο τους πολύ στενούς γείτονές του. Το όριο απόφασης γίνεται πολύ ευέλικτο και μπορεί να καταγράψει ακριβή, τοπικά μοτίβα, αλλά μπορεί να κάνει υπερπροσαρμογή (overfitting).

Τα πλεονεκτήματα των SVMs:

- Αποτελεσματικό σε χώρους υψηλών διαστάσεων.
- Εξακολουθεί να είναι αποτελεσματικό σε περιπτώσεις όπου ο αριθμός των διαστάσεων είναι μεγαλύτερος από τον αριθμό των δειγμάτων.
- Χρησιμοποιεί ένα υποσύνολο σημείων εκπαίδευσης στη συνάρτηση απόφασης (που ονομάζονται διανύσματα υποστήριξης), επομένως είναι επίσης αποδοτικό στη μνήμη.
- Μπορούν να καθοριστούν διαφορετικές συναρτήσεις πυρήνα για τη συνάρτηση απόφασης.

Τα μειονεκτήματα των SVMs:

- Εάν ο αριθμός των χαρακτηριστικών είναι πολύ μεγαλύτερος από τον αριθμό των δειγμάτων, πρέπει να αποφευχθεί η υπερπροσαρμογή (**overfitting**) στην επιλογή των συναρτήσεων πυρήνα και να δοθεί προσοχή στον όρο **regularization**.
- Τα SVM δεν παρέχουν άμεσα εκτιμήσεις πιθανοτήτων, αυτές υπολογίζονται χρησιμοποιώντας μια ακριβή πενταπλάσια διασταυρούμενη επικύρωση.
- Λειτουργούν καλύτερα σε μικρά σετ δειγμάτων λόγω του μεγάλου χρόνου προπόνησής του.

PCA: Η ανάλυση κύριου συστατικού (PCA) είναι μια τεχνική γραμμικής μείωσης διαστάσεων. Μετατρέπει ένα σύνολο δεδομένων με πολλά χαρακτηριστικά σε ένα μικρότερο σύνολο κύριων στοιχείων, ενώ διατηρεί όσο το δυνατόν περισσότερη διακύμανση (πληροφορίες).

Kernel trick: Το κόλπο του πυρήνα είναι μια τεχνική που επιτρέπει να εφαρμόζονται γραμμικά μοντέλα σε μη γραμμικά προβλήματα μετατρέποντας τα δεδομένα σε ένα χώρο χαρακτηριστικών υψηλότερης διάστασης. Χρησιμοποιείται συχνά στη μηχανική μάθηση για προγνωστική μοντελοποίηση, όπως ταξινόμηση, παλινδρόμηση, ομαδοποίηση και μείωση διαστάσεων.

Στην παρούσα εργασία δοκιμάστηκαν όλα τα μοντέλα πυρήνα, για διάφορες τιμές στον καθένα. Ο κώδικας υλοποιήθηκε στο προγραμματιστικό περιβάλλον jupyter notebook. Για την εκπαίδευση του μοντέλου χρησιμοποιήθηκε το σύνολο δεδομένων MNIST, το οποίο σχετίζεται με την αναγνώριση ψηφίων/αριθμών. Περιέχει 70.000 εικόνες ψηφίων από 0-9 (σύνολο 10 κλάσεις) σε κλίμακα greyscale, εκ των οποίων οι 60.000 είναι εικόνες εκπαίδευσης και οι 10.000 εικόνες δοκιμής. Η ανάλυσή τους είναι 28x28 pixels, δηλαδή 784 pixel ανά εικόνα (784 χαρακτηριστικά). Αυτά τα χαρακτηριστικά απεικονίζονται ως ένα μονοδιάστατο διάνυσμα, δηλαδή κάθε χαρακτηριστικό είναι «απλωμένο» το ένα δίπλα στο άλλο δημιουργώντας ένα μονοδιάστατο πίνακα. Τα labels είναι αριθμοί από το 0-9, αντικατοπτρίζοντας το εικονιζόμενο ψηφίο.

CIFAR-10:

Έπειτα το τελικό μοντέλο δοκιμάστηκε και στο σύνολο δεδομένων CIFAR-10. Το σύνολο δεδομένων CIFAR-10 αποτελείται από 50.000 εικόνες εκπαίδευσης διάστασης 32*32*3, επειδή εδώ είναι 3 κανάλια χρώματος (RGB), και από 10.000 εικόνες δοκιμών. Οι κλάσεις διαχωρισμού, όπως και οι ετικέτες που αντικατοπτρίζουν την εκάστοτε εικόνα είναι : "Airplane", "Automobile", "Bird", "Cat", "Deer", "Dog", "Frog", "Horse", "Ship", "Truck".

ΥΛΟΠΟΙΗΣΗ ΚΑΙ ΕΠΕΞΗΓΗΣΗ ΚΩΔΙΚΑ **MNIST**

Αρχικά, έγινε φόρτωση των απαραίτητων βιβλιοθηκών για τη δημιουργία ενός πλαισίου για την υλοποίηση ενός αγωγού μηχανικής μάθησης για την ταξινόμηση του συνόλου δεδομένων MNIST χρησιμοποιώντας Μηχανές Διανυσμάτων Υποστήριξης (SVM).

```
import numpy as np

from tensorflow.keras.datasets import mnist

from sklearn.svm import SVC

from sklearn.model_selection import GridSearchCV, train_test_split

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix, ConfusionMatrixDisplay

from sklearn.decomposition import PCA

#from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt
```

Φόρτωση του συνόλου δεδομένων MNIST και εμφάνιση των δεδομένων για τον έλεγχο σωστής φόρτωσης των δεδομένων. Εξαγόμενο αποτέλεσμα: Training data shape: (60000, 28, 28) (60000,), Αυτό υποδεικνύει ότι το σύνολο δεδομένων εκπαίδευσης αποτελείται από 60000 εικόνες μεγέθους 28*28, όπως ήταν αναμενόμενο και 60000 ετικέτες, όπου η καθεμία αντιστοιχεί σε μία εικόνα.

Test data shape: (10000, 28, 28) (10000,), Αυτό υποδεικνύει ότι το σύνολο δεδομένων δοκιμών αποτελείται από 10000 εικόνες μεγέθους 28*28 και 10000 ετικέτες, όπου η καθεμία αντιστοιχεί σε μία εικόνα.

```
# Load MNIST dataset from Keras

(train_data, train_labels), (test_data, test_labels) = mnist.load_data()



# Display shapes of the datasets

print("Training data shape:", train_data.shape, train_labels.shape)

print("Test data shape:", test_data.shape, test_labels.shape)
```

Αυτός ο κώδικας επιλέγει τυχαία 9 εικόνες από το σύνολο test dataset MNIST και τις εμφανίζει σε ένα πλέγμα 3×3 με τις πραγματικές τους ετικέτες ως τίτλους/ετικέτες.

```
# Randomly sample some images from the test dataset

np.random.seed(42)  # For reproducibility

number_of_samples = 9  # Number of images to display

random_samples = np.random.randint(0, test_data.shape[0],
size=(number_of_samples,))


# These lines extract the randomly selected images and their labels based on
the randomly generated indices.

random_test_images = test_data[random_samples]

random_label_images = test_labels[random_samples]


# Create a subplot grid

fig, ax = plt.subplots(nrows=3, ncols=3, figsize=(7, 7))

cntr = 0

for i in range(3):

   for j in range(3):

      ax[i, j].imshow(random_test_images[cntr], cmap='gray')  # Display image in
grayscale

      ax[i, j].axis("off")  # Turn off axis

      ax[i, j].set_title(f"True: {random_label_images[cntr]}", fontsize=10)  # Show
true label

      cntr += 1

plt.show()
```

Το παρακάτω τμήμα κώδικα μετατρέπει τις εικόνες (και από τα δύο σύνολα δεδομένων) από δύο διαστάσεις (28*28) σε μονοδιάστατα διανύσματα μήκους 784 στοιχείων. Αυτή η μετατροπή είναι απαραίτητη επειδή οι μηχανές διανυσμάτων υποστήριξης (SVM), αναμένουν τα δεδομένα εισόδου να έχουν τη μορφή διανυσμάτων και όχι πολυδιάστατων πινάκων.

```
# Flatten the images from 28x28 to 784

train_data_flattened = train_data.reshape(train_data.shape[0], -1)  # Flatten each image to a vector

test_data_flattened = test_data.reshape(test_data.shape[0], -1)  # Flatten each image to a vector
```

Έπειτα εκτελείται κανονικοποίηση του συνόλου δεδομένων MNIST κλιμακώνοντας τις τιμές των εικονοστοιχείων από το εύρος [0, 255] στο [0, 1]. Ο στόχος της κανονικοποίησης είναι να αλλάξει τις τιμές των αριθμητικών στηλών στο σύνολο δεδομένων για να χρησιμοποιηθεί μια κοινή κλίμακα, χωρίς να παραμορφωθούν οι διαφορές στο εύρος τιμών ή να χαθούν πληροφορίες. Διασφαλίζει ότι όλα τα χαρακτηριστικά (εντάσεις εικονοστοιχείων) συμβάλλουν εξίσου στο μοντέλο. Χωρίς κανονικοποίηση, χαρακτηριστικά με μεγαλύτερα εύρη θα μπορούσαν να κυριαρχούν σε άλλα, οδηγώντας σε κακή σύγκλιση ή χαμηλή απόδοση μοντέλου. Έπειτα γίνεται εκτύπωση των δεδομένων για να επαληθευτεί ότι η κανονικοποίηση έγινε σωστά. Επίσης, τα σύνολα δεδομένων training και test έχουν την μορφή διανύσματος, (60000, 784) και (10000, 784) αντίστοιχα. Το κάθε στοιχείο αντιπροσωπεύεται ως διάνυσμα 784 χαρακτηριστικών.

```
# Normalize the data (scale pixel values to [0, 1])

train_data_flattened = train_data_flattened.astype('float32') / 255.0

test_data_flattened = test_data_flattened.astype('float32') / 255.0


# Print the range of values after normalization

printf("Range of normalized training data: min = {train_data_flattened.min()}, max = {train_data_flattened.max()}")

printf("Range of normalized test data: min = {test_data_flattened.min()}, max = {test_data_flattened.max()}")


printf("Shape of training data after flattening: {train_data_flattened.shape}")

printf("Shape of test data after flattening: {test_data_flattened.shape}")
```

Παρακάτω, εφαρμόζεται Principal Component Analysis (PCA) και στα δύο σύνολα δεδομένων για να μειωθεί η διάστασή τους διατηρώντας παράλληλα το 95% της διακύμανσης στα δεδομένα. Το 95% καθορίζει ότι ο αλγόριθμος PCA πρέπει να διατηρεί το 95% της διακύμανσης στο σύνολο δεδομένων και θα καθορίσει αυτόματα τον βέλτιστο αριθμό των principal components που απαιτούνται για την κάλυψη αυτού του ορίου διακύμανσης. Η επιλογή της διατήρησης της διακύμανσης 95% στην PCA είναι μια κοινή ευρετική λύση. Οι χαμηλότερες τιμές (π.χ. 90%) ενδέχεται να χάσουν ορισμένες κρίσιμες πληροφορίες, ενώ οι υψηλότερες τιμές (π.χ. 99%) διατηρούν πάρα πολλά στοιχεία, ακυρώνοντας τον σκοπό της μείωσης διαστάσεων. Έπειτα, πραγματοποιήθηκε εκτύπωση των δεδομένων και το εξαγόμενο αποτέλεσμα υποδεικνύει ότι έγινε όντως μείωση διάστασης:

Shape of training data after PCA: (60000, 154)

Shape of test data after PCA: (10000, 154)

```
# Apply PCA to reduce dimensionality, retaining 95% of the variance

pca = PCA(n_components=0.95)

train_data_pca = pca.fit_transform(train_data_flattened)

test_data_pca = pca.transform(test_data_flattened)


printf("Shape of training data after PCA: {train_data_pca.shape}")

printf("Shape of test data after PCA: {test_data_pca.shape}")
```

Σε αυτό το σημείο, ο κώδικας εκπαιδεύει έναν ταξινομητή Support Vector Machine (SVM) χρησιμοποιώντας έναν πυρήνα (σε αυτή την περίπτωση γραμμικό – linear) στα δεδομένα εκπαίδευσης που έχουν μετασχηματιστεί από PCA. Οι πυρήνες επιλέγονται βάσει της πολυπλοκότητας του εκάστοτε προβλήματος ταξινόμησης που χρειάζεται να λυθεί, και ανάλογα με τον πυρήνα, χρειάζονται και οι απαραίτητες υπερπαράμετροι. Στην περίπτωση του SVM, μία από τις βασικές υπερπαράμετροι είναι η C (παράμετρος Regularization). Η παράμετρος C βοηθά στην μεγιστοποίηση του περιθωρίου μεταξύ του ορίου απόφασης (διαχωρίζοντας υπερεπίπεδο) και των σημείων δεδομένων, και στην ελαχιστοποίηση σφαλμάτων ταξινόμησης (λανθασμένη ταξινόμηση παραδειγμάτων εκπαίδευσης). Μια υψηλότερη τιμή C δίνει μεγαλύτερη έμφαση στην ελαχιστοποίηση των σφαλμάτων ταξινόμησης, γεγονός που μπορεί να οδηγήσει σε υπερβολική προσαρμογή (το μοντέλο προσπαθεί να ταξινομήσει τέλεια τα δεδομένα εκπαίδευσης). Μια χαμηλότερη τιμή C επιτρέπει στο μοντέλο να έχει μεγαλύτερο περιθώριο, αποδέχεται ορισμένα σφάλματα ταξινόμησης, αλλά δυνητικά βελτιώνει τη γενίκευση. Μια πολύ υψηλή τιμή, όπου το μοντέλο εστιάζει σε μεγάλο βαθμό στην ελαχιστοποίηση των σφαλμάτων ταξινόμησης, τα οποία θα μπορούσαν να οδηγήσουν σε υπερβολική προσαρμογή.

```
# Create the SVM model

svm = SVC(kernel='linear', C=1, random_state=42)

svm.fit(train_data_pca, train_labels)
```

Το μοντέλο προβλέπει τις ετικέτες δεδομένων εκπαίδευσης χρησιμοποιώντας το εκπαιδευμένο μοντέλο SVM και αξιολογεί την ακρίβεια της εκπαίδευσης (training). Η ακρίβεια εκπαίδευσης (training accuracy) θα πρέπει γενικά να είναι λίγο μεγαλύτερη από την ακρίβεια της δοκιμής (test accuracy), επειδή το μοντέλο είναι άμεσα βελτιστοποιημένο για να αποδίδει καλά στα δεδομένα εκπαίδευσης (training data) και λόγω των προκλήσεων της γενίκευσης σε δεδομένα που δεν έχει ελέγξει ακόμα ο αλγόριθμος. Όταν η ακρίβεια της εκπαίδευσης είναι πολύ μεγαλύτερη από αυτήν της δοκιμής, τότε συμβαίνει το φαινόμενο της υπερεκπαίδευσης. Όταν η ακρίβεια της εκπαίδευσης είναι πολύ μικρή, όπως και αυτή της δοκιμής, τότε συμβαίνει το φαινόμενο της υποεκπαίδευσης.

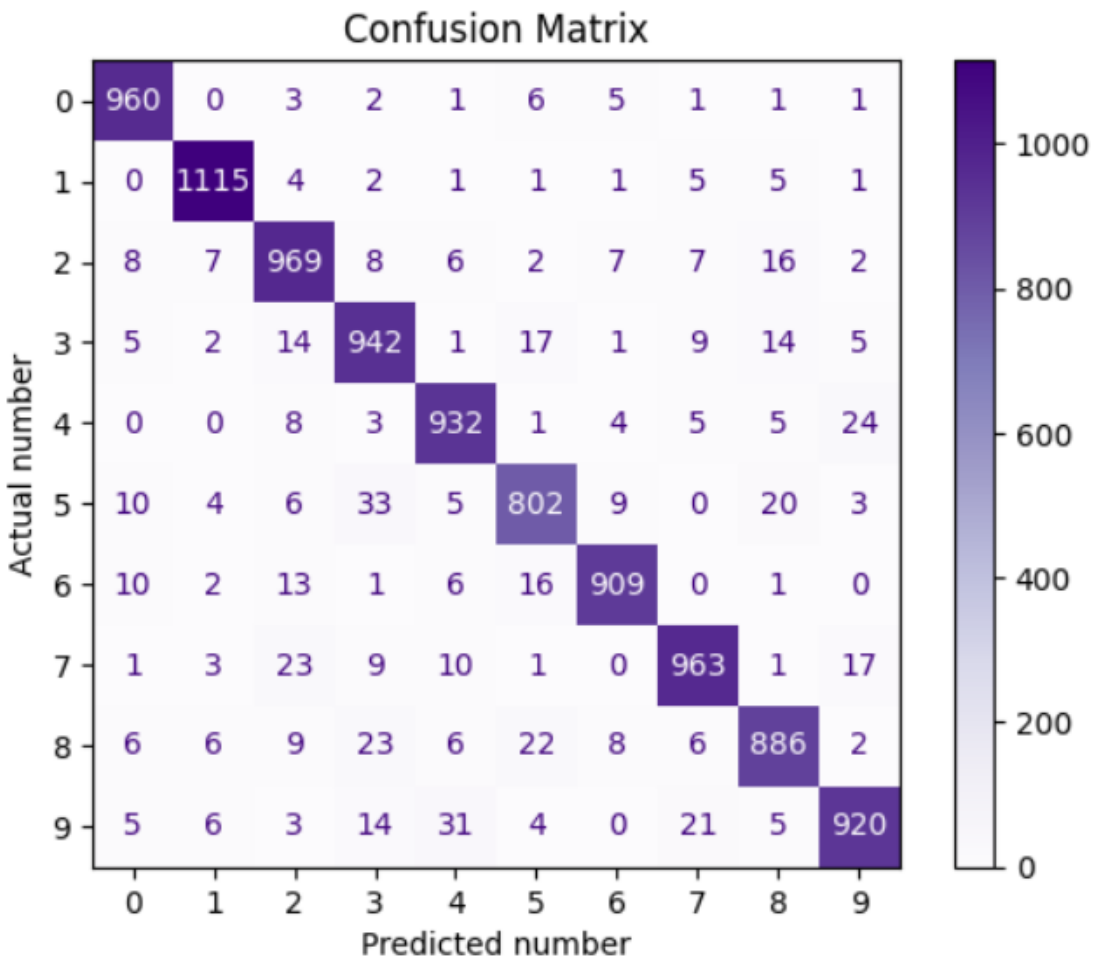
```
#training acc has to be a  little be higher than test acc

train_predictions = svm.predict(train_data_pca) #The model predicts the training data labels using the trained SVM model


# Evaluate training accuracy

train_accuracy = accuracy_score(train_labels, train_predictions) #Compares the predicted labels (train_predictions) to the true (train_labels) and calculates the accuracy

print(f"Training Accuracy: {train_accuracy:.4f}")
```

Με την ακρίβεια δοκιμής, φαίνεται πόσο καλά έχει εκπαιδευτεί το μοντέλο σε νέα δεδομένα, καθώς εδώ υπάρχει και η πρόκληση τα νέα δεδομένα να μην είναι τόσο ευανάγνωστα (γενικά να υπάρχει θόρυβος), και ελέγχεται σε τι ποσοστό οι ετικέτες έχουν αντιστοιχηθεί σωστά στις εικόνες.

```
# Make predictions on the test set

test_predictions = svm.predict(test_data_pca)


# Evaluate the model

accuracy = accuracy_score(test_labels, test_predictions)

print(f"Test Accuracy: {accuracy:.4f}")
```

Το ακόλουθο τμήμα κώδικα δημιουργεί και οπτικοποιεί τον πίνακα σύγχυσης, ο οποίος παρέχει μια λεπτομερή ανάλυση της απόδοσης ταξινόμησης του μοντέλου σε όλες τις κατηγορίες (ψηφία 0–9). Οι γραμμές του πίνακα αντιστοιχούν στις αληθινές ετικέτες, ενώ οι στήλες αντιστοιχούν στις προβλεπόμενες ετικέτες. Διαγώνια βρίσκονται τα στοιχεία που προβλέφθηκαν και ήταν τελικά σωστά, δηλαδή η ετικέτα κάθε εικόνας ανταποκρίθηκε σωστά στο περιεχόμενό της. Ακολουθεί ένα ενδεικτικό παράδειγμα, καθώς ο πίνακας σύγχυσης αλλάζει ανάλογα με τον πυρήνα και τις υπερπαραμέτρους.

```
# Confusion Matrix

conf_matrix = confusion_matrix(test_labels, test_predictions)

# Display the confusion matrix

plt.figure(figsize=(10, 10))

disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix,
display_labels=np.arange(10))

disp.plot(cmap=plt.cm.Purples)

plt.xlabel('Predicted number')

plt.ylabel('Actual number')

plt.title("Confusion Matrix")

plt.show()
```



Εικόνα 2: Πίνακας σύγχυσης για kernel=linear, c=50

ΥΛΟΠΟΙΗΣΗ ΚΑΙ ΕΠΕΞΗΓΗΣΗ ΚΩΔΙΚΑ **CIFAR-10**

Ο κώδικας του CIFAR-10 με τον κώδικα του συνόλου δεδομένου MNIST που αναφέρθηκε προηγουμένως παραμένει ίδιος εκτός από συγκεκριμένες αλλαγές. Οι αλλαγές αυτές αφορούν την βιβλιοθήκη και την φόρτωση του CIFAR-10, όπως και την επιπλέον πληροφορία των ονομάτων των κλάσεων. Επίσης, οι δύο κώδικες διαφέρουν στην οπτικοποίηση των δεδομένων, καθώς ο MNIST περιέχει μόνο εικόνες στην κλίμακα του γκρι, ενώ ο CIFAR-10 έγχρωμες (RGB). Γι' αυτό και στην διαδικασία μετατροπής των εικόνων από

τριών διαστάσεων που ήταν αρχικά σε μονοδιάστατα διανύσματα, γίνεται ο υπολογισμός 32*32*3=3072. Τελική διαφορά των δύο κωδίκων έγκειται στον πίνακα σύγχυσης, καθώς πλέον υπάρχουν άλλα ονόματα κλάσεων σε σχέση με πριν. Οι διαφορές στα τμήματα κώδικα που αναφέρθηκαν αναγράφονται παρακάτω με μπλε χρώμα.

```
import numpy as np #numerical operations and handling arrays

from tensorflow.keras.datasets import cifar10

from sklearn.svm import SVC #for support vector Classification.

#hyperparameter tuning using grid search and cross-validation.

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix, ConfusionMatrixDisplay #evaluate performance

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt #visualizing data/results
```

```
# Load CIFAR-10 dataset from Keras

(train_data, train_labels), (test_data, test_labels) = cifar10.load_data()

# Display shapes of the datasets

print("Training data shape:", train_data.shape, train_labels.shape)

print("Test data shape:", test_data.shape, test_labels.shape)

class_names = [

   "Airplane", "Automobile", "Bird", "Cat", "Deer",

   "Dog", "Frog", "Horse", "Ship", "Truck"

]
```

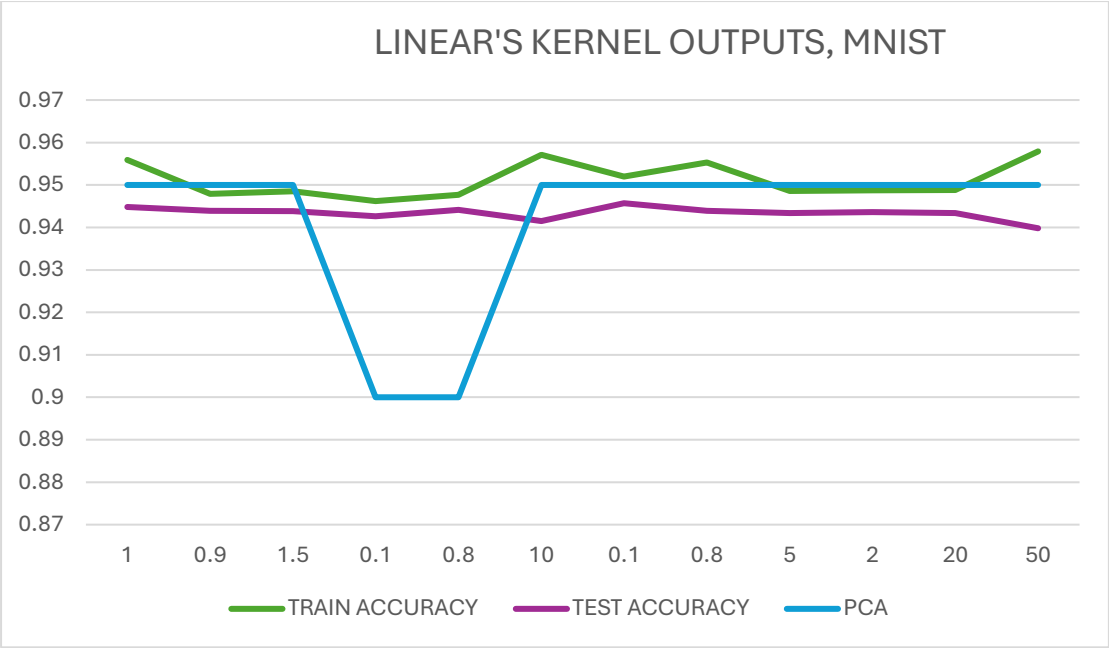
```
# Randomly sample some images from the test dataset

np.random.seed(42)  # For reproducibility

number_of_samples = 9  # Number of images to display

random_samples = np.random.randint(0, test_data.shape[0],
size=(number_of_samples,))


# These lines extract the randomly selected images and their labels based on
the randomly generated indices.

#random_test_images = test_data[random_samples]

#random_test_labels = test_labels[random_samples]


# These lines extract the randomly selected images and their labels based on
the randomly generated indices.

random_test_images = test_data[random_samples]

random_label_images = test_labels[random_samples]


# Create a subplot grid

fig, ax = plt.subplots(nrows=3, ncols=3, figsize=(10, 10))

for i, axi in enumerate(ax.flat):

   axi.imshow(random_test_images[i])  # Display the image in color

   true_label = class_names[random_test_labels[i][0]]  # Convert label index to
class name

   axi.axis("off")  # Turn off axis

   axi.set_title(f"True: {true_label}", fontsize=10)  # Show the true class name

plt.tight_layout()

plt.show()
```

```
# Confusion Matrix

conf_matrix = confusion_matrix(test_labels, test_predictions)


# Display the confusion matrix with class names

plt.figure(figsize=(10, 10))

disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix,
display_labels=class_names) #for cifar10

disp.plot(cmap=plt.cm.Purples, xticks_rotation=45)

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

plt.title("Confusion Matrix")

plt.show()
```

ΑΝΑΛΥΣΗ ΚΑΙ ΧΡΗΣΗ ΔΙΑΦΟΡΕΤΙΚΩΝ ΠΥΡΗΝΩΝ ΣΤΟΝ ΚΩΔΙΚΑ

LINEAR KERNEL, DIFFERENT VALUES OF C

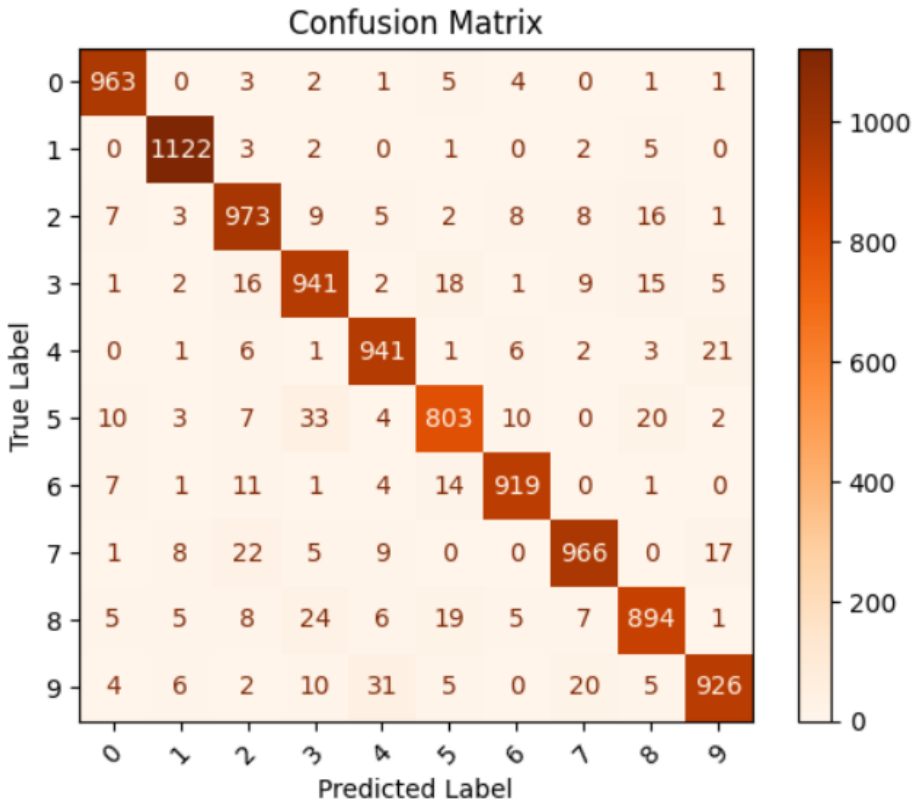
KERNEL	C PARAMETER	TRAIN ACCURACY	TEST ACCURACY	TIME	PCA	DATASET
LINEAR	1	0.9559	0.9448	2m 8.4 sec	0.95	MNIST
LINEAR	0.9	0.9479	0.9439	1m 30.4sec	0.95	MNIST
LINEAR	1.5	0.9485	0.9438	2m 3.9sec	0.95	MNIST
LINEAR	0.1	0.9462	0.9426	58sec	0.90	MNIST
LINEAR	0.8	0.9477	0.9441	1m 35.4sec	0.90	MNIST
LINEAR	10	0.9571	0.9415	11m 1.9sec	0.95	MNIST
LINEAR	0.1	0.9520	0.9457	1m 16.7 sec	0.95	MNIST
LINEAR	0.8	0.9553	0.9439	1m 58.9sec	0.95	MNIST
LINEAR	5	0.9486	0.9434	7m 55.7sec	0.95	MNIST
LINEAR	2	0.9487	0.9436	2m 13.7sec	0.95	MNIST
LINEAR	20	0.9488	0.9434	9m 34 sec	0.95	MNIST
LINEAR	50	0.9579	0.9398	33m 41.9 sec	0.95	MNIST
LINEAR	1	0.4204	0.4080	26m 58.6sec	0.90	CIFAR10



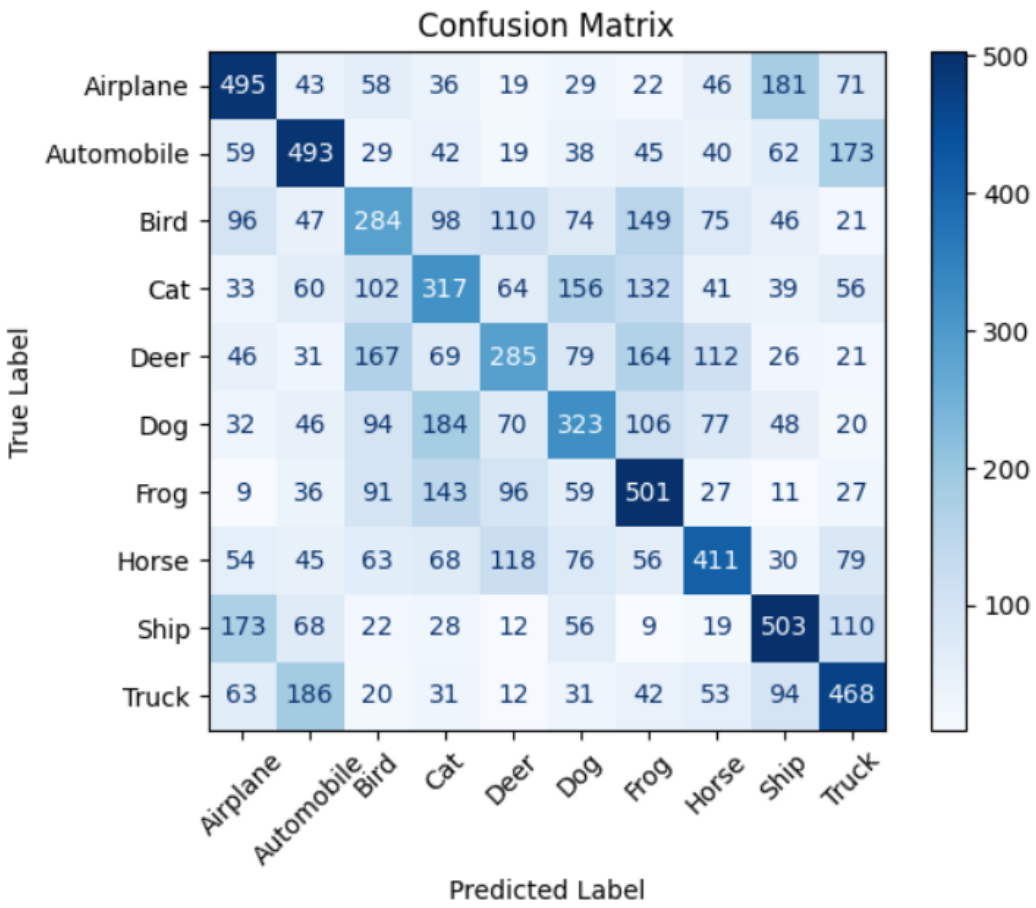
Στον επάνω πίνακα, δοκιμάστηκαν διάφορες τιμές C και PCA στον γραμμικό πυρήνα, για να εξαχθεί το καλύτερο μοντέλο SVM, ώστε να εκπαιδευτεί κατάλληλα και σωστά το σύνολο δεδομένων MNIST. Καταγράφηκαν και παρατηρήθηκαν όλες οι αντλούμενες πληροφορίες για κάθε συνδυασμό, ώστε να εξαχθεί ένα όσο το δυνατόν καλύτερο συμπέρασμα, ως προς την εκπαίδευση και τον απαιτούμενο χρόνο. Η παράμετρος C σε ένα SVM λειτουργεί ως παράμετρος κανονικοποίησης, όπου εάν το C είναι χαμηλό, τα όρια δεν είναι τόσο αυστηρά, επιτρέπει περισσότερες παραβιάσεις περιθωρίων (δηλαδή σφάλματα), οδηγώντας σε ένα απλούστερο μοντέλο με μεγαλύτερο περιθώριο. Αυτό μπορεί να οδηγήσει σε έλλειψη προσαρμογής, καθώς το μοντέλο δίνει προτεραιότητα στην απλότητα έναντι της ακρίβειας. Όσο υψηλότερο είναι το C, δεν επιτρέπει τις παραβιάσεις των περιθωρίων, αυξάνεται δηλαδή η πολυπλοκότητα του μοντέλου, οδηγώντας σε στενότερο περιθώριο γύρω από τα διανύσματα υποστήριξης, κάτι που συνήθως οδηγεί σε υπερπροσαρμογή, και απαιτείται και περισσότερος χρόνος. Για να είναι σωστά προσαρμόσιμο ένα μοντέλο, χωρίς να κάνει υπερπροσαρμογή, ή υποπροσαρμογή, πρέπει το training accuracy να είναι ελαφρώς υψηλότερο από το test accuracy. Το PCA ορίστηκε στο τελικά στο 95%, επειδή προσέφερε υψηλότερα ποσοστά ακρίβειας, ώστε να διατηρηθεί το 95% την πληροφορίας. Παρατηρείται ότι ο καλύτερος συνδυασμός για την εκπαίδευση του μοντέλου είναι αυτός με C=1, train accuracy:0.9559, test accuracy: 0.9448, time: 2m 8.4sec. Το ποσοστό train accuracy και test accuracy έχουν καλή διαφορά μεταξύ τους, και ο χρόνος για να τρέξει το μοντέλο για τα συγκεκριμένα αποτελέσματα είναι πολύ καλός. Για μεγαλύτερο C (αύξηση), τα ποσοστά ακρίβειας είναι ελάχιστα καλύτερα, και ο χρόνος αρκετά πιο πολύς, οπότε οι

συγκεκριμένες παράμετροι προσφέρουν καλό ποσοστό ακρίβειας συγκριτικά με τον χρόνο.

Έπειτα, η καλύτερη επιλογή για το σύνολο δεδομένων MNIST (C=1, train accuracy:0.9559, test accuracy: 0.9448, time: 2m 8.4sec, PCA = 95%), δοκιμάστηκε και για το σύνολο δεδομένων CIFAR-10, το οποίο είναι μεγαλύτερο dataset και περισσοτέρων διαστάσεων, αφού περιέχει εικόνες RGB (σε αντίθεση με τις grey-scale). Σε αυτή την περίπτωση το μοντέλο αναμένεται αν μην είναι τόσο αποδοτικό, γεγονός που αποδεικνύεται με τα εξαγόμενα αποτελέσματα. Για kernel=linear και C=1, οι ακρίβεια των αποτελεσμάτων που προκύπτει από την εκπαίδευση του μοντέλου είναι train accuracy=0.4202, test accuracy=0.4080 και time=26m 58.6sec. Το μοντέλο έχει υποεκπαιδευτεί, διότι τα στατιστικά της ακρίβειας είναι πολύ χαμηλά, και ο χρόνος αρκετά μεγάλος, αφού το μοντέλο δυσκολεύεται να εκπαιδευτεί στα δεδομένα. Αυτό σημαίνει ότι ο γραμμικός πυρήνας δεν είναι ικανός να καταγράψει την πολυπλοκότητα του συνόλου δεδομένων CIFAR-10, αφού είναι πολύ απλός για να χειριστεί τέτοιες μη γραμμικές σχέσεις στα δεδομένα. Η πολυπλοκότητα του CIFAR-10, προκύπτει από τα κανάλια χρώματος, από τα δύσκολα μοτίβα στις εικόνες και από τον αριθμό των εικόνων. Το CIFAR-10 ιδανικά θα χρειαστεί πιο ισχυρούς πυρήνες που να ανταπεξέρχονται στη μη-γραμμικότητα για μεγάλο πλήθος εικόνων και για πολλές κλάσεις.



Εικόνα 3: Πίνακας σύγχυσης για βέλτιστη επίδοση,kernel=linear, C=1, PCA=95%, MNIST



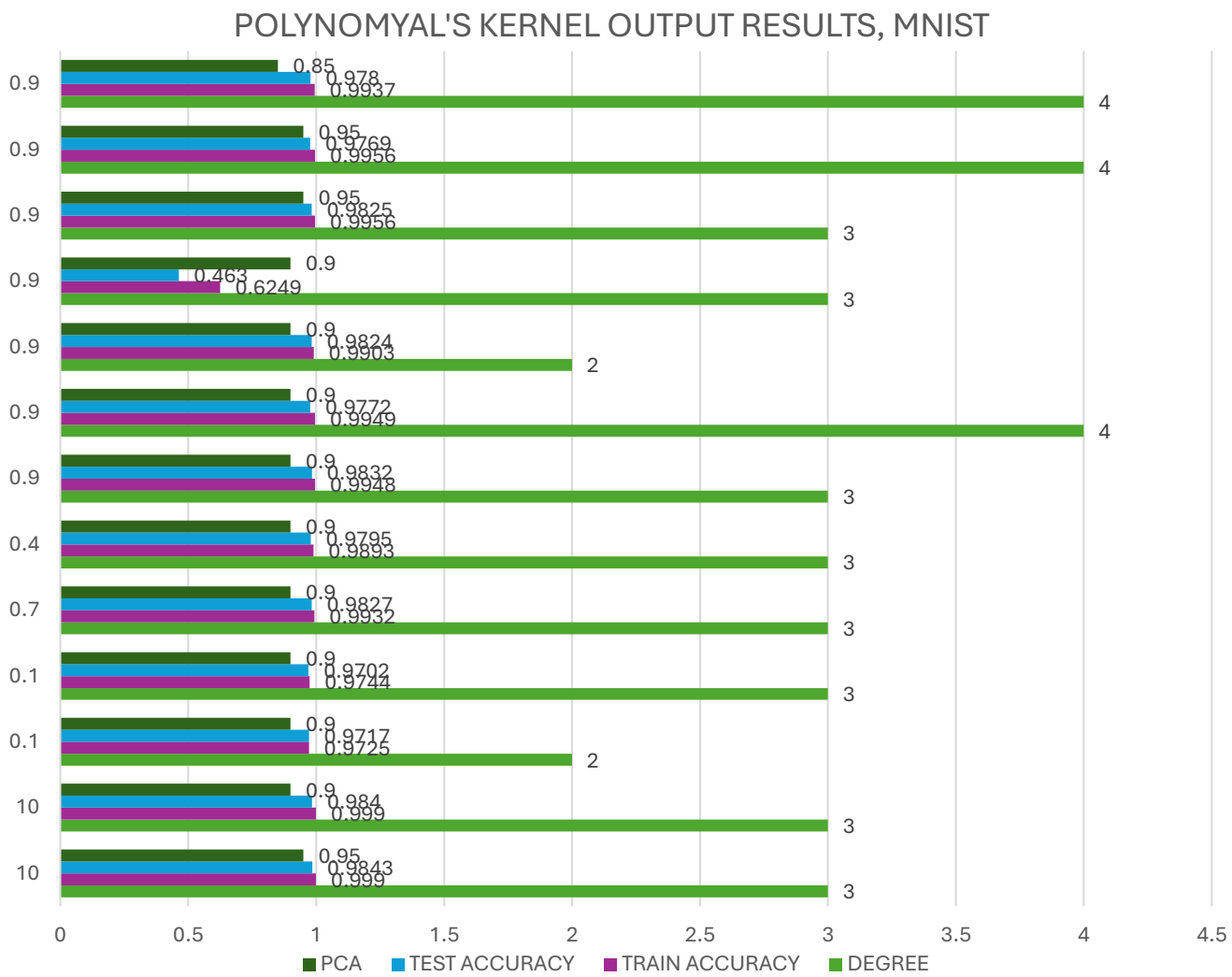
Εικόνα 4: Πίνακας σύγχυσης για βέλτιστη επίδοση,kernel=linear, C=1, PCA=95%, CIFAR10

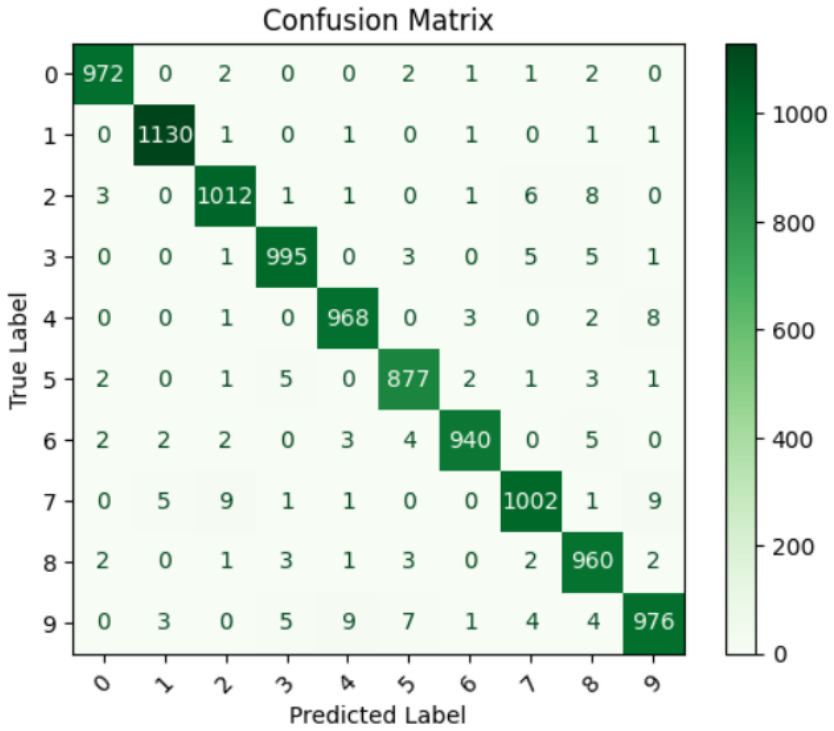
POLYNOMYAL KERNEL, DIFFERENT VALUES OF C, DEGREE

KERNEL	C PARAMETER	DEGREE	TRAIN ACCURACY	TEST ACCURACY	TIME	PCA	DATASET
POLYNOM	10	3	0.9235	0.4859	22m 50sec	0.95	CIFAR10
POLYNOM	10	3	0.999	0.9843	2m 7.2sec	0.95	MNIST
POLYNOM	10	3	0.999	0.984	1m 25.3 sec	0.9	MNIST
POLYNOM	0.1	2	0.9725	0.9717	2m 50.4 sec	0.9	MNIST
POLYNOM	0.1	3	0.9744	0.9702	3m 32.3 sec	0.9	MNIST
POLYNOM	0.7	3	0.9932	0.9827	1m 57sec	0.9	MNIST
POLYNOM	0.4	3	0.9893	0.9795	2m 8.6 sec	0.9	MNIST
POLYNOM	0.9	3	0.9948	0.9832	1m 42 sec	0.9	MNIST
POLYNOM	0.9	4	0.9949	0.9772	2m 52 sec	0.9	MNIST
POLYNOM	0.9	2	0.9903	0.9824	1m 50.6sec	0.9	MNIST
POLYNOM	0.9	3	0.6249	0.4630	10m 1.6sec	0.9	CIFAR10
POLYNOM	0.9	3	0.9956	0.9825	2m 39.1sec	0.95	MNIST
POLYNOM	0.9	4	0.9956	0.9769	6m 1.3sec	0.95	MNIST
POLYNOM	0.9	4	0.9937	0.9780	2m 22.3sec	0.85	MNIST
POLYNOM	10	3	0.8791	0.4796	10m 45.1sec	0.9	CIFAR10

Ο βαθμός του πολυωνύμου καθορίζει την υψηλότερη ισχύ του πολυωνύμου. Για παράδειγμα, αν degree=2, ο πυρήνας θα εξετάσει τις τετραγωνικές αλληλεπιδράσεις μεταξύ των χαρακτηριστικών. Η προσαρμογή αυτής της παραμέτρου επιτρέπει τον έλεγχο για το πόσο πολύπλοκο γίνεται το όριο απόφασης μεταξύ των κλάσεων. Ο βαθμός επηρεάζει σημαντικά την απόδοση, συγκεκριμένα, οι χαμηλότεροι βαθμοί (π.χ. 2) αποδίδουν γενικά χαμηλότερες ακρίβειες από τον βαθμό 3, που εξισορροπεί την πολυπλοκότητα και την απόδοση. Οι υψηλότεροι βαθμοί (όπως ο 4) δείχνουν ελαφρώς μειωμένη

ακρίβεια δοκιμής σε σύγκριση με τον βαθμό 3, λόγω υπερβολικής προσαρμογής. Ο χρόνος αυξάνεται με υψηλότερους βαθμούς, καθώς οι πολυωνυμικοί υπολογισμοί μεγαλώνουν σε πολυπλοκότητα. Η περίπτωση με C=0.9, βαθμό=3, PCA=0.9 είναι η βέλτιστη με train accuracy=0.9948, test accuracy=0.9832 και χρόνο εκπαίδευσης του μοντέλου:1m 42 sec. Ένα πολυώνυμο χαμηλότερου βαθμού εξασφαλίζει καλύτερη γενίκευση σε αόρατα δεδομένα. Παρατηρώ επίσης, ότι για μικρότερα C η ακρίβεια είναι μικρότερη, γεγονός που οφείλεται στο γεγονός ότι επιτρέπονται περισσότερα σφάλματα. Για C=10, το test accuracy είναι ελάχιστα υψηλότερο, αλλά χρειάζεται αρκετά περισσότερο χρόνο, γι'αυτό και η επιλογή C=0.9 είναι η βέλτιστη, διότι παρέχει τον καλύτερο και πιο ακριβή συνδυασμό. Με τη χρήση PCA=0.9, απορρίπτεται περισσότεροςθόρυβος ή λιγότερο σημαντικές πληροφορίες στο σύνολο δεδομένων, αφήνοντας μόνο τα πιο σημαντικά χαρακτηριστικά. Αυτό μειώνει τον κίνδυνο υπερπροσαρμογής του μοντέλου, ειδικά με τον πολυωνυμικό πυρήνα, ο οποίος μπορεί να δημιουργήσει πολύ περίπλοκα όρια απόφασης καθώς αυξάνεται ο βαθμός και ο C. Τα δεδομένα χαμηλότερης διάστασης (PCA=0.9) επεξεργάζονται ταχύτερα, γι' αυτό και οι χρόνοι εκπαίδευσης για PCA=0.9 είναι γενικά μικρότεροι σε σύγκριση με το PCA=0.95. Οι πολυωνυμικοί πυρήνες είναι πολύπλοκοι, ειδικά για υψηλότερους βαθμούς και μεγαλύτερες τιμές C, και τείνουν να υπερπροσαρμόζουν. Το PCA=0.9 έχει καλύτερη απόδοση εδώ επειδή μειώνει πιο άμεσα τον θόρυβο και τις διαστάσεις, δεν υπερπροσαρμόζει όπως θα γινόταν με PCA=0.95. Δηλαδή επειδή ο πολυωνυμικός πυρήνας τείναι να υπερπροσαρμόζει, όπως και το υψηλό PCA, το χαμηλό PCA αποτελεί καλύτερο συνδυασμό. Έπειτα, οι καλύτερες παράμετροι δοκιμάστηκαν και στο σύνολο δεδομένων CIFAR-10, με τα αποτελέσματα να υποδεικνύουν υπερεκπαίδευση train accuracy=0.6249, test accuracy=0.4630, time=10m 1.6sec, αφού το test accuracy είναι πολύ μικρότερο από το train accuracy. Ο πυρήνας ταιριάζει καλά στα δεδομένα εκπαίδευσης, αλλά δεν μπορεί να γενικευτεί στα δεδομένα δοκιμής. Ο πολυωνυμικός πυρήνας καταγράφει ορισμένες μη γραμμικές σχέσεις στο CIFAR10, σε σχέση με τον γραμμικό πυρήνα, αλλά υπολείπεται σε γενίκευση. Για μεγαλύτερες τιμές C, παρατηρείται υπερεκπαίδευση.

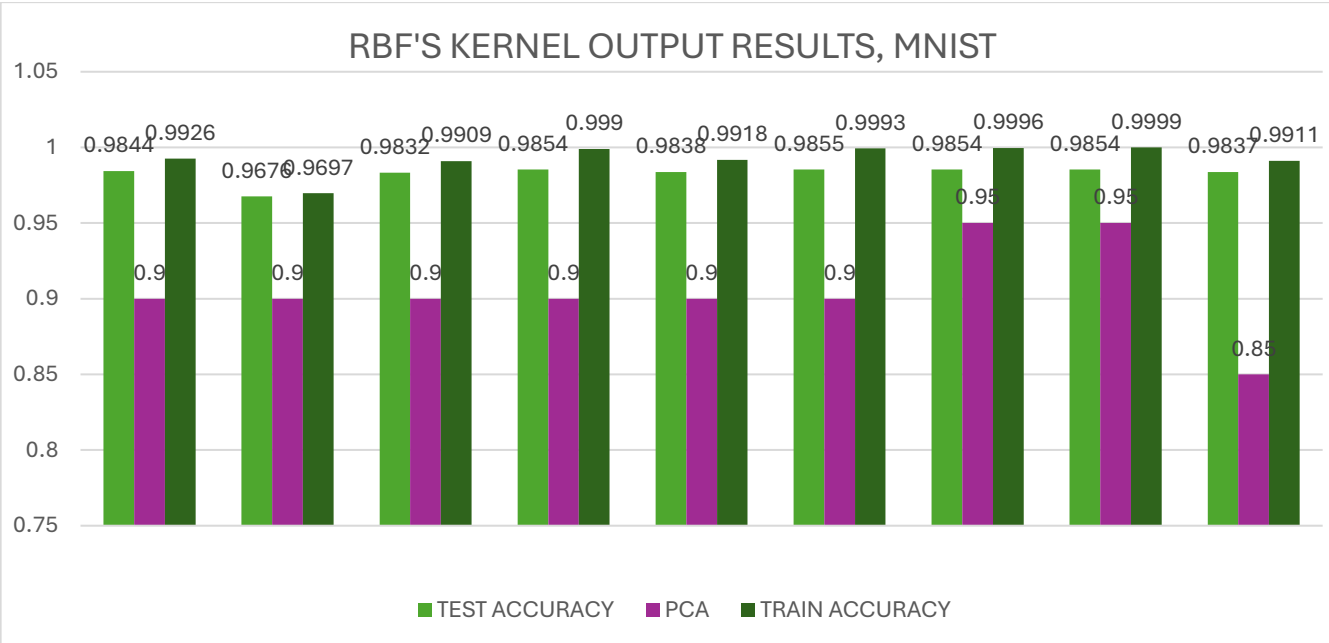




Εικόνα5:kernel=polynomyal, C=0.9, gamma=scale, PCA=0.9, MNIST

RBF KERNEL, DIFFERENT VALUES OF C, GAMMA=SCALE

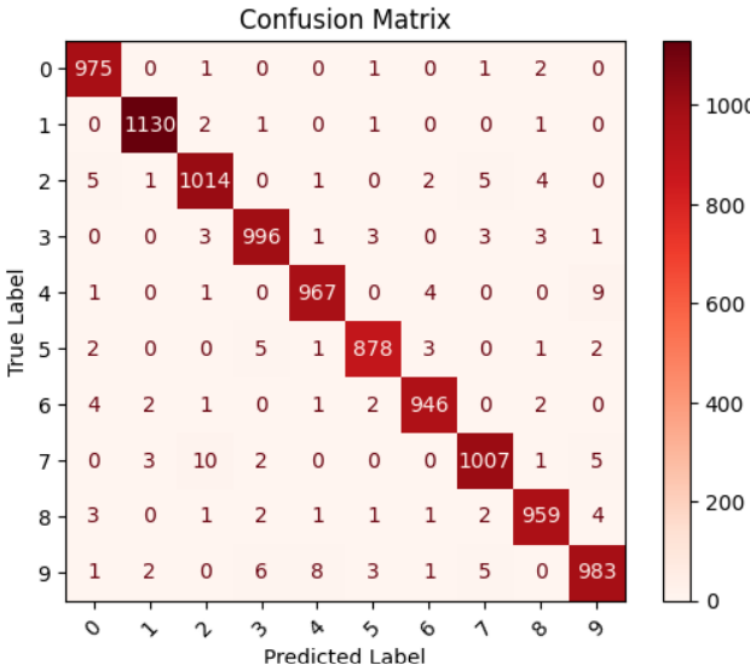
KERNEL	C PARAMETER	GAMMA	TRAIN ACCURACY	TEST ACCURACY	TIME	PCA	DATASET
RBF	1	scale	0.9926	0.9844	1m 8.6sec	0.9	MNIST
RBF	0.1	scale	0.9697	0.9676	2m17.4 sec	0.9	MNIST
RBF	0.8	scale	0.9909	0.9832	1m 5.0sec	0.9	MNIST
RBF	10	scale	0.999	0.9854	1m 1.7sec	0.9	MNIST
RBF	0.9	scale	0.9918	0.9838	1m 8.8sec	0.9	MNIST
RBF	5	scale	0.9993	0.9855	1m 1.1sec	0.9	MNIST
RBF	5	scale	0.8528	0.5618	10m 20.6sec	0.9	CIFAR10
RBF	5	scale	0.9996	0.9854	2m 4.7sec	0.95	MNIST
RBF	10	scale	0.9999	0.9854	2m 4.2sec	0.95	MNIST
RBF	0.9	scale	0.9911	0.9837	55.1 sec	0.85	MNIST
RBF	20	scale	0.9644	0.5555	14m 11.9sec	0.9	CIFAR10
RBF	10	scale	0.9216	0.5624	15m 8.6sec	0.9	CIFAR10



Παρατηρώ, ότι όπως και πριν, η καλύτερη απόδοση είναι για PCA=0.9, και οι καλύτερες παράμετροι είναι C=5, gamma=scale με αποτελέσματα: train accuracy=0.993, test accuracy=0.9855 και time=1m 1.1sec. Ο πυρήνας RBF είναι εξαιρετικά ευέλικτος και προσαρμόζεται σε περίπλοκα όρια αποφάσεων. Λειτουργεί ιδιαίτερα καλά σε σύνολα δεδομένων με περίπλοκα και υψηλών διαστάσεων μοτίβα όπως το MNIST και το CIFAR10. Οι χαμηλές τιμές C επιτρέπουν στο μοντέλο να γενικεύεται καλύτερα και να επιτρέπει λάθη. Οι υψηλότερες τιμές C επιτρέπουν στο μοντέλο να δημιουργήσει πολύ συγκεκριμένα όρια, οδηγώντας σε τέλεια ακρίβεια εκπαίδευσης αλλά συνήθως υπερπροσαρμογή. Η παράμετρος gamma καθορίζει την επίδραση μεμονωμένων παραδειγμάτων εκπαίδευσης. Όταν gamma='scale', η τιμή του gamma υπολογίζεται αυτόματα ως:

gamma=1/ (num of features * variance of features) = ‘scale’ parameter

Αυτή η κανονικοποίηση διασφαλίζει ότι το gamma δεν είναι ούτε πολύ μεγάλο ούτε πολύ μικρό, γεγονός που βοηθά στην αποφυγή υπερπροσαρμογής ή υποπροσαρμογής. Προσαρμόζεται στη διακύμανση και τη διάσταση των δεδομένων. Οι υψηλές τιμές gamma καθιστούν το όριο απόφασης πολύ ευαίσθητο σε μικρές διακυμάνσεις στα δεδομένα (υπερπροσαρμογή). Τα εξαγόμενα αποτελέσματα για υψηλότερα C, ενώ θα ήταν αναμενόμενο να είναι καλύτερα, λόγω υπερπροσαρμογής, δεν είναι τόσο καλά. Αυτό μπορεί να συμβαίνει επειδή το μοντέλο δέχεται και αρκετό θόρυβο στην προσπάθειά του να επεξεργαστεί και να μάθει όσο το δυνατόν καλύτερα τα δεδομένα.



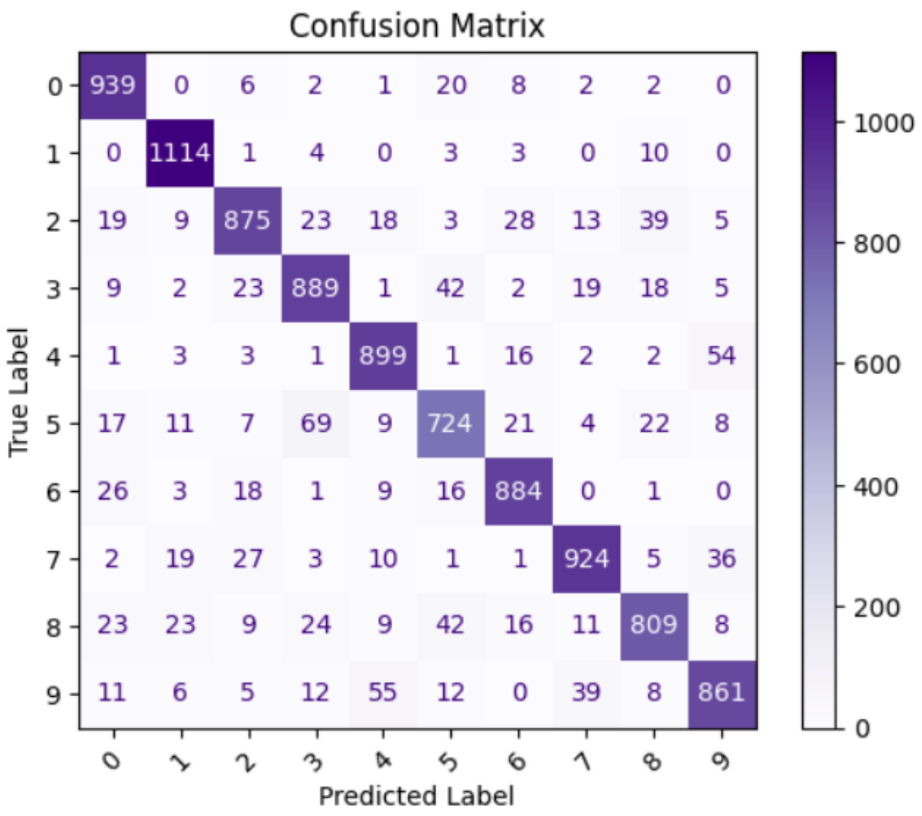
Εικόνα 5:kernel=RBF, gamma=scale, C=5, PCA=0.9,MNIST

Το CIFAR-10 δοκιμάστηκε στα ίδια δεδομένα, (C=5, gamma=scale) με train accuracy=0.8528, test accuracy=0.5618 και time=10m 20.6sec. Ο πυρήνας RBF υπερτερεί τόσο του πολυωνυμικού όσο και του σιγμοειδούς και του γραμμικού πυρήνα στο CIFAR-10. Η ικανότητα του πυρήνα RBF να μοντελοποιεί μη γραμμικά όρια αποφάσεων τον καθιστά κατάλληλο για το σύνολο δεδομένων CIFAR-10. Ο χρόνος εκπαίδευσης (10m 20.6 sec) σε σύγκριση με τον πολυωνυμικό πυρήνα είναι περίπου ο ίδιος, ωστόσο είναι πιο αποτελεσματικός δεδομένης της σημαντικά καλύτερης ακρίβειάς του. Επιτυγχάνει υψηλότερη ακρίβεια δοκιμής (56.18%), υποδηλώνοντας καλύτερη γενίκευση. Επίσης, δεν έχει υπερεκπαιδευτεί επειδή η διαφορά της ακρίβειας εκπαίδευσης και δοκιμών δεν είναι τόσο μεγάλη, για ένα τόσο περίπλοκο σύνολο δεδομένων. Ο πυρήνας RBF με gamma=scale προσαρμόζει δυναμικά την τιμή gamma με βάση τον χώρο χαρακτηριστικών του συνόλου δεδομένων. Αυτό εμποδίζει τον πυρήνα να γίνει πολύ ευαίσθητος σε μεμονωμένα σημεία δεδομένων και μειώνει τον κίνδυνο υπερπροσαρμογής. Η παράμετρος κανονικοποίησης C=5 είναι αρκετά μεγάλη ώστε να επιτρέπει στο μοντέλο να καταγράφει περίπλοκα μοτίβα αλλά όχι υπερβολικά υψηλή. Όταν το C αυξάνεται C=5πολύ (C=10, 20), παρατηρείται υπερπροσαρμογή. Επομένως το C=5, είναι όντως η καλύτερη επιλογή.

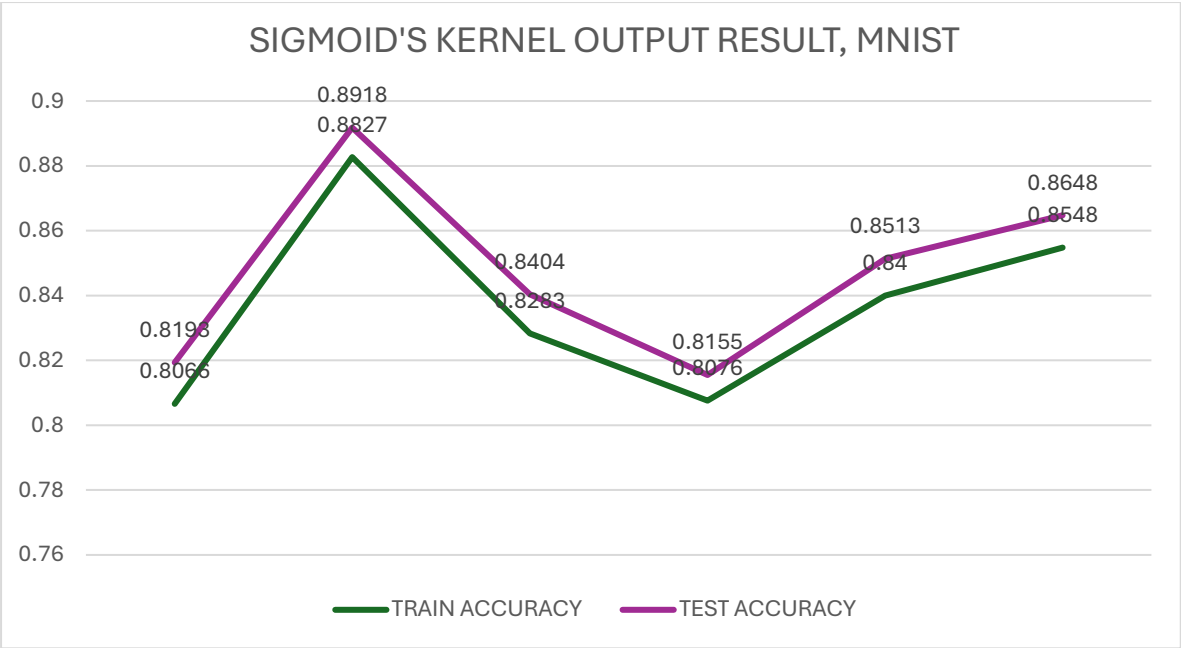
SIGMOID KERNEL, DIFFERENT VALUES OF C

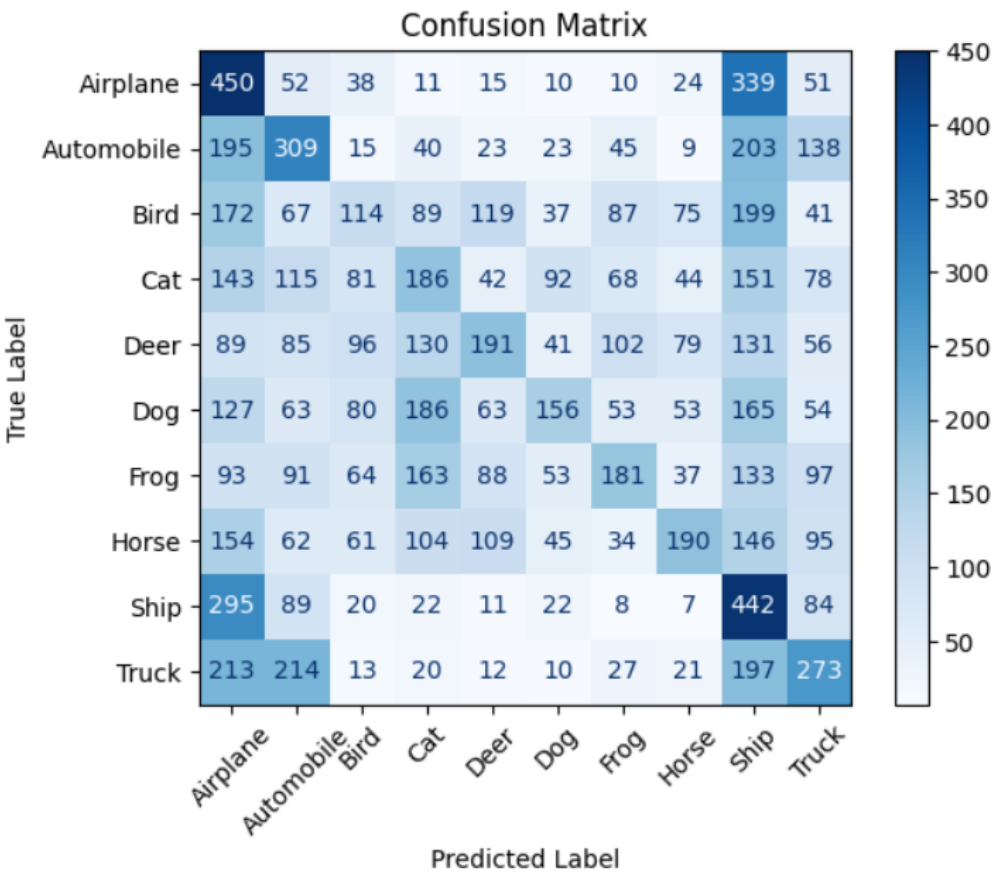
KERNEL	C PARAMETER	TRAIN ACCURACY	TEST ACCURACY	TIME	PCA	DATASET
SIGMOID	10	0.8066	0.8193	1m 14sec	0.9	MNIST
SIGMOID	0.1	0.8827	0.8918	2m 26.8sec	0.9	MNIST
SIGMOID	0.8	0.8283	0.8404	1m 20sec	0.9	MNIST
SIGMOID	50	0.8076	0.8155	1m 4.3sec	0.9	MNIST
SIGMOID	0.5	0.84	0.8513	1m 40.8sec	0.9	MNIST
SIGMOID	0.5	0.8548	0.8648	2m 51.3 sec	0.95	MNIST
SIGMOID	0.1	0.2503	0.2492	9m 50.8sec	0.9	CIFAR10

Παρατηρώ ότι καλύτερο αποτέλεσμα παρέχει ο sigmoid kernel C=0.1, με train accuracy=0.8827, test accuracy=0.8918 και time= 2m 26.8sec. Γενικά, ο σιγμοειδής πυρήνας, είναι πιο αποδοτικός σε μικρά σύνολα δεδομένων με περιορισμένη μη-γραμμικότητα. Γι' αυτό και στη συγκεκριμένη περίπτωση δεν παρέχει καλά αποτελέσματα. Τα αποτελέσματα που έχουν προκύψει υποδηλώνουν έναν όχι τόσο αποδοτικό πυρήνα για τα συγκεκριμένα σύνολα δεδομένων. Όσο και να αυξάνεται το C, η ακρίβεια παραμένει χαμηλή. Ο sigmoid kernel με C=0.1 εξήγαγε αποτελέσματα train accuracy=0.2503, test accuracy=0.2492 και time=9m 50.8sec. Είναι το χαμηλότερο ποσοστό ακρίβειας και ο ίδιος χρόνος, συγκριτικά με τους υπόλοιπους δοκιμασμένους πυρήνες, καθιστώντας τον πιο αδύναμο να ανταπεξέλθει σε αυτό το σύνολο δεδομένων, αφού δεν μπορεί να μάθει τα μοτίβα. Ο σιγμοειδής πυρήνας είναι ακατάλληλος για το CIFAR10 λόγω της αδυναμίας του να χειριστεί την πολυπλοκότητα του συνόλου δεδομένων και λόγω της εξαιρετικά χαμηλής ακρίβειας που προκύπτει.



Εικόνα 6: kernel=sigmoid, C=0.1, PCA=0.9, MNIST



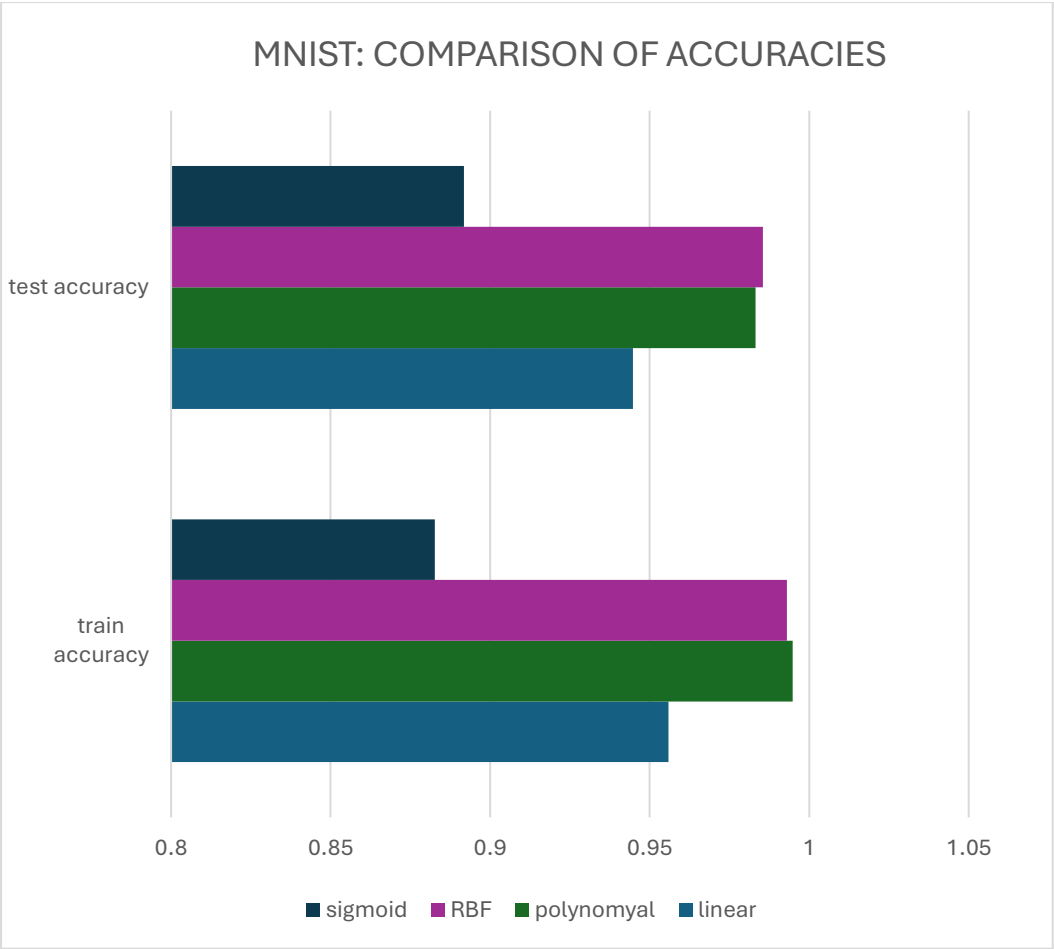


Εικόνα 7: kernel=sigmoid, C=0.1, PCA=0.9, CIFAR-10

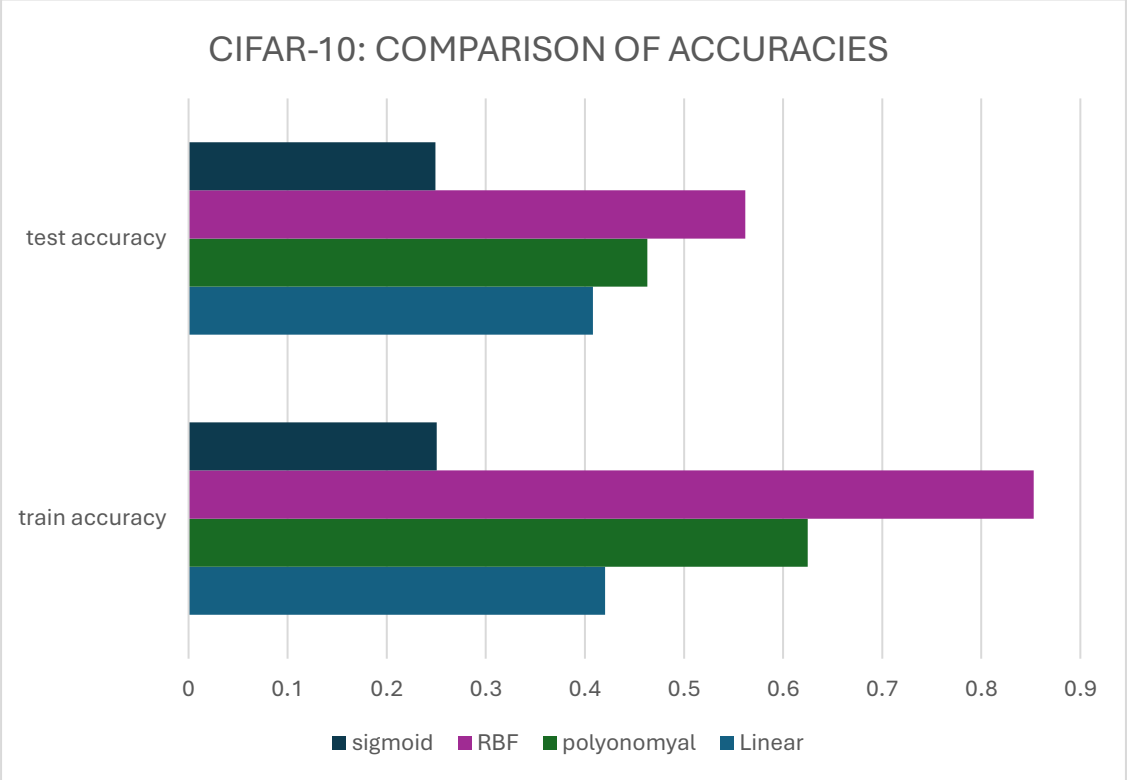
ΣΥΓΚΡΙΣΗ ΚΑΙ ΕΥΡΕΣΗ ΤΟΥ ΚΑΛΥΤΕΡΟΥ ΠΥΡΗΝΑ ΓΙΑ ΤΟ **MNIST**

KERNEL	C PARAMETER	DEGREE - GAMMA	TRAIN ACCURACY	TEST ACCURACY	TIME	PCA	DATASET
LINEAR	1	-	0.9559	0.9448	2m 8.4 sec	0.95	MNIST
POLYNOM	0.9	3	0.9948	0.9832	1m 42 sec	0.9	MNIST
RBF	5	scale	0.9993	0.9855	1m 1.1sec	0.9	MNIST
SIGMOID	0.1	-	0.8827	0.8918	2m 26.8sec	0.9	MNIST

Παρατηρείται ότι ο αποδοτικότερος πυρήνας είναι ο RBF με τις μεταβλητές και τα αποτελέσματα να αναγράφονται στον επάνω πίνακα. Πέτυχε σταθερά υψηλή ακρίβεια εκπαίδευσης και δοκιμής, αποδεικνύοντας την ικανότητά του να μαθαίνει αποτελεσματικά από το σύνολο δεδομένων. Μπορεί να μοντελοποιήσει σύνθετα όρια αποφάσεων, τα οποία είναι απαραίτητα για σύνολα δεδομένων όπως το MNIST. Αυτός ο πυρήνας επέδειξε ισχυρή ικανότητα γενίκευσης, καθώς όχι μόνο πέτυχε υψηλή ακρίβεια εκπαίδευσης (99.93%) αλλά διατήρησε επίσης μια σχετικά υψηλή ακρίβεια δοκιμής 98.55%. Στο C=5, ο πυρήνας RBF επιτυγχάνει μια καλή ισορροπία μεταξύ ακρίβειας και υπολογιστικού κόστους (1m 1.1s). Ο επόμενος στην σειρά είναι ο πολυωνυμικός, ο οποίος παρουσιάζει επίσης ισχυρή απόδοση, με ακρίβεια εκπαίδευσης 99.48% και ακρίβεια δοκιμής 98.32%. Ωστόσο, η απόδοσή του είναι ελαφρώς χαμηλότερη από τον πυρήνα RBF, καθώς αυτό φαίνεται και στον χρόνο εκπαίδευσής τους. Ο γραμμικός πυρήνας και ο σιγμοειδής αποτελούν τις λιγότερο καλές επιλογές, λόγω του υψηλού χρόνου και της χαμηλής απόδοσης, γεγονός που υποδεικνύει ανικανότητα καλής και γρήγορης εκμάθησης των μοτίβων του συνόλου δεδομένων. Ο πυρήνας RBF ξεπερνά σταθερά τον πολυωνυμικό πυρήνα σε σύνθετα σύνολα δεδομένων λόγω της ικανότητάς του να δημιουργεί ευέλικτα, μη γραμμικά όρια απόφασης. Για το MNIST, ο πυρήνας RBF επιτυγχάνει υψηλή ακρίβεια δοκιμής ακόμη και με υψηλό C επειδή το σύνολο δεδομένων είναι σχετικά καθαρό (χαμηλός θόρυβος).



ΣΥΓΚΡΙΣΗ ΚΑΛΥΤΕΡΩΝ ΔΟΚΙΜΑΣΜΕΝΩΝ ΠΥΡΗΝΩΝ ΓΙΑ ΤΟ **CIFAR-10**

KERNEL	C PARAMETER	DEGREE, GAMMA	TRAIN ACCURACY	TEST ACCURACY	TIME	PCA	DATASET
LINEAR	1	-	0.4204	0.4080	26m 58.6sec	0.90	CIFAR10
POLYNOM	0.9	3	0.6249	0.4630	10m 1.6sec	0.9	CIFAR10
RBF	5	scale	0.8528	0.5618	10m 20.6sec	0.9	CIFAR10
SIGMOID	0.1	-	0.2503	0.2492	9m 50.8sec	0.9	CIFAR10

Από τους συγκεκριμένους αποδοτικότερους πυρήνες που δοκιμάστηκαν στο MNIST dataset, παρατηρείται σαφή διαφορά στο CIFAR-10 dataset. Είναι όλοι τους λιγότερο αποδοτικοί, λόγω της υψηλής περιπλοκότητας του συγκεκριμένου συνόλου δεδομένων (RBG images). Ο καλύτερος πυρήνας για το CIFAR-10 είναι ο RBF, με υπερπαράμετρο C=5, gamma=scale, train accuracy=0.8525, test accuracy=0.5618, time=10m 20.6sec, PCA=0.9. Ο συγκεκριμένος συνδυασμός είναι αποτελεσματικότερος επειδή δεν κάνει ούτε υπερεκπαίδευση, ούτε υποεκπαίδευση (π.χ. σιγμοειδής), παρέχει την καλύτερη ακρίβεια δοκιμών, δηλαδή μπορεί να μάθει καλύτερα τα μοτίβα του συνόλου δεδομένων. Ο υπολογιστικός χρόνος των 10m 20.6s είναι λογικός δεδομένης της πολυπλοκότητας του συνόλου δεδομένων και της ικανότητας του πυρήνα να μαθαίνει περίπλοκα χαρακτηριστικά. Ο πυρήνας RBF είναι καλύτερος για το CIFAR10 επειδή μοντελοποιεί αποτελεσματικά τις μη γραμμικές σχέσεις του συνόλου δεδομένων, ενώ εξισορροπεί κάπως την ακρίβεια και την υπολογιστική απόδοση. Ο πολυωνυμικός πυρήνας περιορίζεται από την υπερπροσαρμογή και τη χαμηλότερη ακρίβεια δοκιμής, καθιστώντας τον δευτερεύουσα επιλογή, καθώς η ακρίβεια είναι χαμηλότερη, ενώ ο γραμμικός και ο σιγμοειδής δεν έχουν καλή απόδοση για αυτό το σύνολο δεδομένων και θα πρέπει να αποφευχθούν. Το CIFAR-10 αποτελείται από εξαιρετικά πολύπλοκα δεδομένα εικόνας, ένα γραμμικό όριο απόφασης δεν επαρκεί για να καταγράψει τα εγγενή μοτίβα και τις σχέσεις στα δεδομένα , γι' αυτό και ο γραμμικός πυρήνας δεν αρκεί.